

**МИНИСТЕРСТВО НАУКИ И ВЫСШЕГО ОБРАЗОВАНИЯ
РОССИЙСКОЙ ФЕДЕРАЦИИ**

**ФЕДЕРАЛЬНОЕ ГОСУДАРСТВЕННОЕ БЮДЖЕТНОЕ ОБРАЗОВАТЕЛЬНОЕ
УЧРЕЖДЕНИЕ ВЫСШЕГО ОБРАЗОВАНИЯ
«ВЯТСКИЙ ГОСУДАРСТВЕННЫЙ УНИВЕРСИТЕТ»**

Институт математики и информационных систем

Факультет автоматики и вычислительной техники

Кафедра электронных вычислительных машин

Отчёт по проекту
«Weather Forecast»
по дисциплине
«Программирование»

Выполнил студент гр. ИВТ6-3302-05-00

_____/Иванов И.И./
(Подпись)

Проверил профессор кафедры ЭВМ

_____/Петров П.П./
(Подпись)

Киров
2026

Цель работы

Разработать приложение для автоматического получения прогноза погоды, сохранения его в базе данных и формирования Markdown-отчёта.

Задание

Изучить и описать архитектуру приложения **Weather Forecast**, реализованного на языке Python. Приложение должно:

- определять текущую локацию пользователя по IP-адресу;
- запрашивать прогноз погоды на 4 дня через OpenWeatherMap API;
- агрегировать 3-часовые данные в дневные записи;
- сохранять данные в SQLite без дубликатов;
- формировать отчёт в формате Markdown;
- выводить сводку в консоль.

Теоретические сведения

В проекте используются следующие технологии и библиотеки:

- **Python 3.10+** — язык программирования, поддерживающий современные конструкции (аннотации типов, `dataclasses`).
- **SQLAlchemy 2.0** — ORM для работы с базой данных SQLite. Позволяет описывать модели как классы Python и выполнять запросы через сессии.
- **requests** — библиотека для выполнения HTTP-запросов к внешним API.
- **python-dotenv** — загрузка переменных окружения из файла `.env`.
- **OpenWeatherMap API** — внешний сервис для получения прогноза погоды. Используется endpoint `/data/2.5/forecast`, возвращающий данные с шагом 3 часа.
- **IP-геолокация** — сервис `ip-api.com` для определения города и координат по IP-адресу.
- **Markdown** — текстовый формат для формирования отчёта.

Ход работы

Архитектура приложения

Приложение состоит из нескольких модулей, каждый из которых отвечает за свою часть функциональности:

- `env_manager.py` — загрузка и валидация переменных окружения, настройка логирования.
- `geoloc.py` — определение локации (IP-геолокация или фолбэк по городу).
- `weather_api.py` — запрос прогноза и агрегация данных по дням.
- `db_manager.py` — подключение к БД, модель `Weather`, сохранение записей.
- `report.py` — формирование Markdown-отчёта.
- `main.py` — точка входа, управляющая последовательностью вызовов.

Взаимодействие модулей можно описать следующей схемой:

`main → geoloc → weather_api → db_manager → report`

Модуль env_manager.py

Модуль загружает переменные из `.env` с помощью `python-dotenv` и предоставляет их остальным частям программы. Обязательные переменные проверяются функцией `_require`. Также настраивается базовое логирование.

```
1 import logging
2 import os
3 from dotenv import load_dotenv
4
5 load_dotenv()
6
7 LEVEL = os.getenv("LOG_LEVEL", "WARNING").upper()
8 logging.basicConfig(level=LEVEL,
9                     format="%(asctime)s [%(levelname)s] %(name)s: %(message)s")
10 log = logging.getLogger(__name__)
11
12 def _require(name: str) -> str:
13     v = os.getenv(name)
14     if v is None:
15         raise RuntimeError(f"{name} is not set")
16     return v
17
18 DB_URL: str = _require("DB_URL")
19 DB_USER: str = os.getenv("DB_USER", "")
20 DB_PASSWORD: str = os.getenv("DB_PASSWORD", "")
21 WEATHER_API_KEY: str = _require("WEATHER_API_KEY")
22 WEATHER_API_URL: str = _require("WEATHER_API_BASE_URL")
23 GEO_API_URL: str | None = os.getenv("GEO_API_URL")
24 GEO_API_FALLBACK_CITY: str | None = os.getenv("GEO_API_FALLBACK_CITY")
25 OUTPUT_FILE: str = os.getenv("OUTPUT_FILE", "weather_report.md")
```

Listing 1: Фрагмент env_manager.py

Модуль geoloc.py

Определяет локацию пользователя. Если задан `GEO_API_URL`, выполняется запрос к сервису IP-геолокации. При успехе возвращается объект `Location` с городом и координатами. В случае ошибки или отсутствия URL используется запасной город из `GEO_API_FALLBACK_CITY`.

```
1 @dataclass
2 class Location:
3     city: str
4     lat: float | None = None
5     lon: float | None = None
6
7 def get_location() -> Location:
8     if GEO_API_URL:
9         try:
10             r = requests.get(GEO_API_URL, timeout=5)
11             r.raise_for_status()
12             data = r.json()
13             city = data.get("city")
14             lat = data.get("lat") or data.get("latitude")
15             lon = data.get("lon") or data.get("longitude")
16             if city and lat is not None and lon is not None:
17                 return Location(city, float(lat), float(lon))
18         except requests.RequestException as e:
19             log.warning("Request to %s failed: %s", GEO_API_URL, e)
20         except ValueError as e:
21             log.warning("JSON parse error: %s", e)
22     if GEO_API_FALLBACK_CITY:
23         return Location(GEO_API_FALLBACK_CITY)
24     raise RuntimeError("Could not determine city.")
```

Listing 2: Функция get_location в geoloc.py

Модуль weather_api.py

Содержит две основные функции: `fetch_forecast` для запроса к OpenWeatherMap и `aggregate_daily` для агрегации 3-часовых данных в дневные.

```
1 def fetch_forecast(loc: Location) -> list[dict]:
2     params = {"appid": WEATHER_API_KEY, "units": "metric", "lang": "ru"}
3     if loc.lat is not None and loc.lon is not None:
4         params["lat"] = loc.lat
5         params["lon"] = loc.lon
6     else:
7         params["q"] = loc.city
8     r = requests.get(f"{WEATHER_API_URL}/forecast", params=params, timeout=10)
9     if r.status_code == 404:
10         raise RuntimeError(f"City not found: {loc.city}")
11     r.raise_for_status()
12     return r.json()["list"]
13
14 def aggregate_daily(items: list[dict], days: int = 4) -> list[dict]:
15     by_day = defaultdict(list)
16     for item in items:
17         d = datetime.fromisoformat(item["dt_txt"]).date()
18         by_day[d].append(item)
19     today = datetime.now(tz=timezone.utc).date()
20     result = []
21     for d in sorted(by_day):
22         if d < today:
23             continue
24         if len(result) >= days:
25             break
26         entries = by_day[d]
27         temps_min = [e["main"]["temp_min"] for e in entries]
28         temps_max = [e["main"]["temp_max"] for e in entries]
29         humidity = [e["main"]["humidity"] for e in entries]
30         wind = [e["wind"]["speed"] for e in entries]
31         descriptions = [e["weather"][0]["description"] for e in entries]
32         result.append({
33             "date": d,
34             "temp_min": min(temps_min),
35             "temp_max": max(temps_max),
36             "humidity": int(sum(humidity) / len(humidity)),
37             "wind_speed": max(wind),
38             "description": max(set(descriptions), key=descriptions.count),
39         })
40     return result
```

Listing 3: Функции `fetch_forecast` и `aggregate_daily`

Модуль db_manager.py

Отвечает за подключение к базе данных, описание модели `Weather` и сохранение записей. Используется уникальное ограничение по паре (`city`, `forecast_date`), чтобы избежать дублирования.

```
1 class Weather(Base):
2     __tablename__ = "weather_forecast"
3     __table_args__ = (
4         UniqueConstraint("city", "forecast_date", name="uq_city_forecast_date"),
5     )
6     id = Column(Integer, primary_key=True)
7     city = Column(String(100), nullable=False)
8     forecast_date = Column(Date, nullable=False)
9     temp_min = Column(Float)
10    temp_max = Column(Float)
11    humidity = Column(Integer)
```

```

12     wind_speed = Column(Float)
13     description = Column(String(100))
14     created_at = Column(DateTime)
15
16 def save_forecast(city: str, rows: list[dict]) -> list[Weather]:
17     existing = {d for (d,) in session.query(Weather.forecast_date)
18                 .filter(Weather.city == city).all()}
19     saved = []
20     for row in rows:
21         if row["date"] in existing:
22             continue
23         w = Weather(city=city, forecast_date=row["date"],
24                    temp_min=row["temp_min"], temp_max=row["temp_max"],
25                    humidity=row["humidity"], wind_speed=row["wind_speed"],
26                    description=row["description"],
27                    created_at=datetime.now(tz=timezone.utc))
28         session.add(w)
29         saved.append(w)
30     session.commit()
31     return saved

```

Listing 4: Модель Weather и функция save_forecast

Модуль report.py

Формирует Markdown-отчёт с выравниванием колонок. Функция `_render` создаёт таблицу, а `write_markdown` записывает её в файл.

```

1 HEADERS = ["Date", "Min T (C)", "Max T (C)", "Description", "Humidity (%)", "Wind (m/s)"]
2
3 def _row_from(r) -> list[str]:
4     return [str(r.forecast_date), str(round(r.temp_min)), str(round(r.temp_max)),
5           r.description.capitalize(), str(r.humidity), str(round(r.wind_speed))]
6
7 def _render(city: str, rows: list) -> str:
8     if not rows:
9         return f"# Weather forecast\n\nLocation: {city}\n\nNo data.\n"
10    body = [_row_from(r) for r in rows]
11    widths = [len(h) for h in HEADERS]
12    for row in body:
13        for i, cell in enumerate(row):
14            widths[i] = max(widths[i], len(cell))
15    def line(cells):
16        return "|" + " | ".join(c.ljust(widths[i])
17                                for i, c in enumerate(cells)) + " |"
18    separator = "|" + " | ".join("-" * (w + 2) for w in widths) + "|"
19    period = f"{rows[0].forecast_date} - {rows[-1].forecast_date}"
20    out = ["# Weather forecast", f"Location: {city}",
21          f"Period: {period}", "", line(HEADERS), separator]
22    out.extend(line(row) for row in body)
23    return "\n".join(out) + "\n"

```

Listing 5: Функция _render в report.py

Главный модуль main.py

Управляет последовательностью действий: определение локации, запрос прогноза, агрегация, сохранение, выборка из БД, формирование отчёта и вывод сводки.

```

1 def main() -> int:
2     loc = get_location()
3     raw = fetch_forecast(loc)
4     daily = aggregate_daily(raw, days=4)
5     saved = save_forecast(loc.city, daily)

```

```

6  today = datetime.now(tz=timezone.utc).date()
7  period_end = today + timedelta(days=3)
8  all_rows = session.query(Weather).filter(
9      Weather.city == loc.city,
10     Weather.forecast_date >= today,
11     Weather.forecast_date <= period_end
12 ).order_by(Weather.forecast_date).all()
13 write_markdown(loc.city, all_rows)
14 period_start = all_rows[0].forecast_date if all_rows else "-"
15 actual_period_end = all_rows[-1].forecast_date if all_rows else "-"
16 print_summary(loc, period_start, actual_period_end, len(saved), OUTPUT_FILE)
17 return 0

```

Listing 6: Функция main в main.py

Настройка и запуск

Для работы приложения необходимо создать файл `.env` на основе `.env.example` и заполнить его. Основные переменные приведены в таблице ??.

Таблица 1

Переменные окружения

Переменная	Описание
DB_URL	URL базы данных (например, sqlite:///./weather.db)
DB_USER	Имя пользователя БД (для SQLite пусто)
DB_PASSWORD	Пароль БД (для SQLite пусто)
WEATHER_API_KEY	API-ключ OpenWeatherMap (обязателен)
WEATHER_API_BASE_URL	Базовый URL API (https://api.openweathermap.org/data/2.5)
GEO_API_URL	URL сервиса IP-геолокации (https://ip-api.com/json/)
GEO_API_FALLBACK_CITY	Запасной город, если IP-геолокация не сработала
LOG_LEVEL	Уровень логирования (DEBUG, INFO, WARNING, ERROR, CRITICAL)
OUTPUT_FILE	Путь к Markdown-файлу отчёта

Зависимости проекта перечислены в `requirements.txt`:

- requests 2.34.2
- SQLAlchemy 2.0.54
- python-dotenv 1.2.3

Запуск производится командой `python src/main.py` или через скрипт `run.sh`.

Пример вывода

Пример Markdown-отчёта:

Weather forecast

Location: Kirov

Period: 2026-09-28 - 2026-10-01

Date	Min T (C)	Max T (C)	Description	Humidity (%)	Wind (m/s)
-----	-----	-----	-----	-----	-----

2026-09-28	8	9	Clear	91	2	
2026-09-29	5	16	Clear	77	3	
2026-09-30	4	16	Clear	78	2	
2026-10-01	6	15	Cloudy	77	3	

Пример консольной сводки:

```
=====
City:                Frankfurt am Main
Coordinates:         50.0827, 8.6297
Forecast period:     2026-09-28 - 2026-10-01
Saved records:       0
Report file:         weather_report.md
=====
```

Вывод

В ходе выполнения проекта разработано приложение **Weather Forecast**, демонстрирующее модульный подход к построению программ на Python. Освоены работа с внешними API (OpenWeatherMap, ip-api.com), ORM SQLAlchemy для взаимодействия с SQLite, обработка ошибок и логирование, а также формирование отчётов в формате Markdown. Приложение успешно решает задачу автоматического получения и сохранения прогноза погоды.